

PROGRAMAÇÃO MODULAR

INTERFACES E POLIMORFISMO

PROF. JOÃO CARAM

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE

POO TRADICIONAL E HERANÇA

2

- Herança: recurso poderoso para relacionamento entre classes e reutilização de código.
 - Polimorfismo.
- Herança inicialmente tida como maior contribuição do paradigma e, assim, largamente utilizada.

POO TRADICIONAL E HERANÇA

3

- ▣ Mas, pensando bem...
 - ▣ Como fica o encapsulamento?
 - ▣ Como fica o acoplamento?
 - ▣ E se houver necessidade de mudança de comportamento em tempo de execução?
 - “Conversão” não é permitida entre classes-irmãs.

INTERFACE

4

- Interface pública de uma classe: métodos disponíveis (que, naturalmente, devem ser implementados naquela classe)

INTERFACE

5

- ▣ Interface pública de uma classe: métodos disponíveis (que, naturalmente, devem ser implementados naquela classe)
- ▣ “Contrato que define tudo o que uma classe deve fazer se quiser ter um determinado status”.

INTERFACE

6

- ▣ “Contrato que define tudo o que uma classe deve fazer se quiser ter um determinado status”.
- ▣ Especificação de uma interface independente:
 - ▣ Uma ou mais classes “assinariam este contrato”, comprometendo-se a implementar o que foi especificado.

INTERFACE

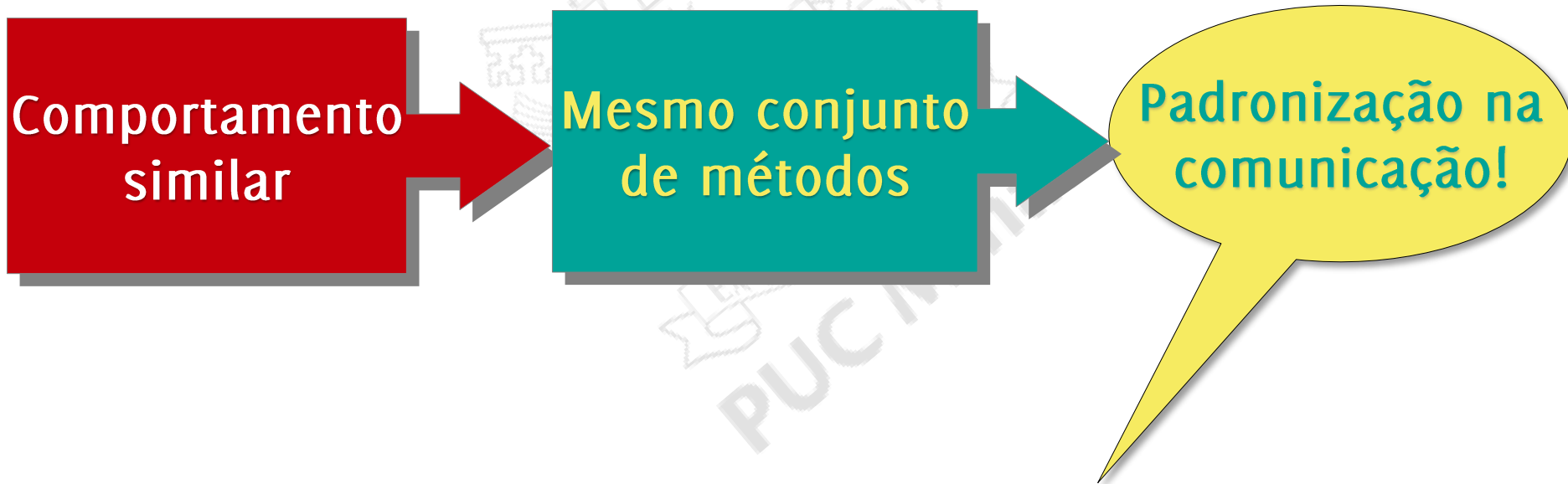
7

- ▣ “Contrato que define tudo o que uma classe deve fazer se quiser ter um determinado status”.
- ▣ Especificação de uma interface independente:
 - ▣ Uma ou mais classes “assinariam este contrato”, comprometendo-se a implementar o que foi especificado.
- ▣ Há um bom motivo para isso?

INTERFACES

8

- Bons motivos: abstração e padronização, favorecendo baixo acoplamento.



INTERFACES

9

- Bons motivos: abstração e padronização, favorecendo baixo acoplamento.
- Outro ótimo motivo: flexibilidade não permitida pela especialização com herança.

INTERFACES

10

- Cenários de uso de interfaces:
 - Habilidades/comportamentos similares em classes de hierarquias diferentes (padronização);
 - Necessidade de múltiplas habilidades em uma classe;
 - Alternativa para implementação da especialização/polimorfismo com menor acoplamento.

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto 'EX CORDE ECCLESIAE', and the text 'PUC Minas' below it.

{ Quero ver código!! }

INTERFACES

12

- ▣ Usadas para definir um protocolo de comportamento que pode ser implementado por qualquer classe em qualquer hierarquia de classes.
- ▣ Não contêm código, somente assinaturas dos métodos relativos a seu comportamento.

INTERFACES

13

- São declaradas, mas não são instanciadas.
- Classes realizam a implementação de interfaces.
 - Obedecem ao protocolo de comportamento.
- Uma classe pode implementar várias interfaces.

INTERFACES EM JAVA

14

- Possuem declarações (assinaturas) de métodos e, se necessário, atributos constantes (*public static final*).
- Podem incorporar outras interfaces, utilizando-se da palavra-chave *extends*.

INTERFACES EM JAVA

15

- ▣ Podem possuir métodos default ou privados.
- ▣ São 'assinadas' usando a palavra-chave implements.

INTERFACES

16

- Aumento da abstração:
 - implementação fica a cargo de cada classe que assina o contrato da interface.
- Aumento da independência:
 - classes são totalmente independentes entre si e de uma possível classe-mãe.

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE